Université de la Manouba
Ecole Nationale des Sciences de L'Informatique



Rapport de stage d'immersion en entreprise

Génération et customisation d'une image Linux pour la cible phytec-mira-imx6 sous l'environnent YOCTO sous DOCKER

Réalisé par :
Assala LAHDHIRI

Organisme : ACTIA Engineering Services (Siège Social)
Encadré par : M. Wajdi ZAIRI
Adresse : Parc Technologique "El-Ghazala", 1 Rue Newton, BP99 - 2088 ARIANA
TEL : 70 687 917



Année Universitaire :
2019-2020

Signature et Appréciations de l'encadrant



Remerciement

Exprimer sa reconnaissance à ceux qui ont collaboré de loin ou de près à l'élaboration de ce projet est un devoir agréable.

Nous présentons donc nos vifs et profonds respects et reconnaissances à notre encadrant Mr. Wajdi ZAIRI, pour avoir accepté de nous superviser et nous diriger lors de la réalisation de ce travail, pour ses conseils et ses compétences qui nous ont considérablement servi pour mener à bien notre application.

Nous remercions également les membres de jury d'avoir eu la bienveillance de nous accorder leur attention et d'évaluer notre présentation.

Finalement, il est indispensable pour nous de remercier sincèrement et révéler notre plus profonde gratitude à nos familles et aux personnes qui nous sont chères pour leur soutien à la fois moral et matériel, ainsi nos amis qui nous ont inspiré à réaliser ce projet et qui nous ont aidé avec leurs conseils et critiques.

Résumé — Ce rapport porte sur la génération et la customisation d'une image d'un système d'exploiation pour la cible phyBOARD-Mira imx6. Cette tâche est réalisé avec Yocto sous Docker pour garantir un maximum de flexibilité ainsi que de la compatibilité et de robustesse pour notre produit final.
Mots clés : Yocto, Linux embarqué, PhyBOARD-Mira imx6, Docker, Dockerfile

Abstract — This report is about building and customizing an OS image for the phyBOARD-Mira imx6 target. This task is done with Yocto under Docker to guarantee a maximum flexibility as well as a compatibility and robustness for our final product.
Key words : Yocto, Embedded Linux, PhyBOARD-Mira imx6, Docker, Dockerfile

ملخص -- يركز هذا التقرير على إنشاء صورة نظام تشغيل وتخصيصها لهدف phyBOARD-Mira imx6.
تتم هذه المهمة باستخدام Yocto تحت Docker لضمان أقصى قدر من المرونة بالإضافة إلى التوافق والمتانة لمنتجنا النهائي.

Table des matières

Introduction Générale **1**

1 État de l'art **3**
1.1 Présentation de l'organisme d'accueil . . . 3
1.1.1 Présentation du groupe ACTIA . . . 3
1.1.2 Présentation de ACTIA Engineering Services . . . 3
1.1.3 Domaines d'activité . . . 4
1.2 Cadre général du projet . . . 4
1.2.1 Étude de l'existant . . . 4
1.2.1.1 La virtualisation et son importance . . . 4
1.2.1.2 Les machines virtuelles . . . 4
1.2.1.3 Buildroot . . . 5
1.2.2 Critique de l'existant . . . 5
1.2.2.1 Crtitique des machines virtuelles . . . 5
1.2.2.2 Critique du Buildroot . . . 5
1.2.3 Solutions proposées . . . 5
1.2.3.1 Docker . . . 5
1.2.3.2 Le Projet Yocto . . . 6

2 Spécification des besoins et conception **8**
2.1 Spécification et analyse des besoins . . . 8
2.1.1 Identification des acteurs . . . 8
2.1.2 Les besoins fonctionnels . . . 8
2.1.3 Les besoins non fonctionnels . . . 9
2.1.4 Diagramme de séquences système . . . 9
2.2 Conception . . . 9
2.2.1 Architecture de Docker . . . 10
2.2.2 Diagramme d'activité du workflow "se connecter, push et pull à partir du Dockerhub" . . . 10

3 Réalisation **12**
3.0.1 PC DELL . . . 12
3.0.2 PhyBOARD-Mira i.MX 6 Full Featured . . . 12
3.1 Environnement logiciel . . . 13
3.1.1 Ubuntu 14.04 LTS . . . 13
3.1.1.1 Ubuntu 16.04 LTS . . . 13
3.1.1.2 Le projet Yocto . . . 13
3.1.1.3 Docker . . . 13
3.2 Travail réalisé . . . 13
3.2.1 Génération et customisation d'une image Linux pour la cible phytec-mira-imx6 sous l'environnent Yocto . . . 13

3.2.2 Sous Docker 15

Table des figures

1.1 Comparaison entre les Conteneurs de Docker et les Machines Virtuelles [7] . . . 6
1.2 Comparaison entre Yocto et Buildroot . . . 6

2.1 L'image sur l'écran lié à la cible . . . 8
2.2 Diagramme Séquences-Système . . . 9
2.3 Architecture Docker [2] . . . 10
2.4 Diagramme d'activité . . . 11

3.1 Le kit PhyBOARD-Mira i.MX 6 Full Featured . . . 12
3.2 headless system packages . . . 13
3.3 tftp-hpa . . . 13
3.4 git config . . . 14
3.5 proxy . . . 14
3.6 wget phyLinux . . . 14
3.7 phyLinux init . . . 14
3.8 imx6 . . . 15
3.9 PD18.1.1 . . . 15
3.10 phyboard-mira-imx6-11 . . . 15
3.11 docker build . . . 15
3.12 successfully built . . . 16
3.13 docker images . . . 16
3.14 docker run root . . . 16
3.15 docker commit . . . 16
3.16 docker run user . . . 16
3.17 . sources . . . 17
3.18 bitbake . . . 17
3.19 UTF-8 erreur . . . 17
3.20 succès bitbake . . . 18
3.21 phyboard-mira-imx6-11 générée . . . 18
3.22 repository vide . . . 18
3.23 docker login . . . 18
3.24 docker tag . . . 19
3.25 docker push . . . 19
3.26 pushing . . . 19
3.27 successfully pushed . . . 19
3.28 repository myfirstproject . . . 20
3.29 docker pull . . . 20

Introduction générale

Dans le domaine de l'embarqué, nous nous retrouvons parfois obligés de reconstruire notre propre système complet à partir des sources, pour notre cible qui est, dans la plupart des cas, différente de notre architecture hôte. L'organisation d'un système embarqué, la compilation (cross-compilation) et les adaptations nécessaires sont des phases fatigantes et lourdes pour tous, débutant ou confirmé soit-on.
Mais heureusement que des outils et des sources libres existent et proposent habituellement des fonctionnalités complémentaires très avantageuses pour simplifier et accélérer cette tâche.
C'est de l'une des ces technologies que nous profitons pour générer et customiser notre propre image pour la cible phyBOARD-Mira i.MX6 pour que cette dernière nous permette de développer davantage des applications avancées à savoir des applications IOT.
Il s'agit effectivement du projet Yocto.

Une autre contrainte qui s'oppose après l'obtention de notre image, est le fait que le développement et de notre application sur un système d'exploitation ou une distribution bien spécifique peut engendrer l'échec de son déploiement et de son exécution sur une autre machine.

D'où notre point de vue qui nous pousse à réaliser tout ce travail dans un Docker container.
Tous ces objectifs réunis sont détaillés dans 3 chapitres :
Le premier chapitre spécifie le cadre du projet, à savoir l'organisme d'accueil ou a eu lieu le stage et son encadrement et l'étude et la critique de l'existant.
Le chapitre qui suit quant à lui est dédié à l'identification des acteurs du systèmes ainsi que des fonctionnalités dont ils peuvent profiter, pour finir avec des diagrammes servant à décrire le système globalement et spécialement.
Et pour finir, le troisième et le dernier chapitre met l'accent sur les détails de la réalisation complète de ce projet.

Chapitre 1

État de l'art

Le long de ce chapitre nous décrivons le cadre et l'organisme d'accueil de ce projet avant de définir le contexte ainsi que les objectifs derrière la réalisation du projet.

1.1 Présentation de l'organisme d'accueil

1.1.1 Présentation du groupe ACTIA

La société ACTIA a été fondée en 1986 par Messieurs, CHABRERIE CALMELS, et PECH dans le cadre du "spin-off" du département "petite série" de BENDIX à Toulouse. ACTIA Group est une entreprise française spécialisée dans la construction des équipements électroniques dont la valeur ajoutée est grande, qui sont destinés aux marchés porteurs de véhicules et de télécommunications. Le group ACTIA est un des leaders dans le domaine des systèmes embarqués, du diagnostic automobile sur les marchés de véhicules en petites et moyennes séries et de la transmission hertzienne. ACTIA groupe réunie deux grandes divisions : la division télécom et la division automotive. Il emploie plus de 2 550 collaborateurs répartis sur 15 pays. Plus de 550 collaborateurs de ACTIA group travaillent en RD dans des bureaux d'études du groupe. Ce dernier a réalisé en 2011 un chiffre d'affaires de 300,4 Millions d'euro.[1]

1.1.2 Présentation de ACTIA Engineering Services

La société ACTIA est une société de Services et de Conseil en Ingénierie, sise au Technopôle Ghazala depuis 2005, régie par la loi 93-120 du code des investissements relative aux sociétés non résidentes et totalement exportatrices. C'est une filiale du groupe mondial français ACTIA, Groupe spécialiste des équipements électroniques à haute valeur ajoutée destinés aux marchés des véhicules roulants et des télécommunications. ACTIA a une forte expérience dans les projets menés à l'échelle internationale. Depuis sa fondation, ACTIA accompagne ses clients dans le développement et l'intégration des systèmes de communication électronique intégrée et logiciels ou progiciels connexes. ACTIA a développé un véritable savoir-faire pour :

— Le développement des logiciels embarqués.

— Les études mécaniques et électroniques.

— Les tests et validation de systèmes complexes.

— Les services d'équipement.

ACTIA a mis en place un système qualité puissant et en constante amélioration, ainsi que des solutions éprouvées. la méthodologie et l'expertise en gestion de projet des meilleures pratiques et des meilleures normes de la industrie du logiciel. ACTIA est certifiée ISO 9001 version 2008 pour toutes ses activités de conception, développement, test et Validation, outils de

diagnostic, systèmes mécaniques, outils de production pour applications automobiles, support industriel et test de qualification. Accrédité ISO 17025 version 2005 depuis 2013 pour ses laboratoire de qualification thermomécanique.

1.1.3 Domaines d'activité

— Automobile et transport.
— IT.
— Industrie.
— Multimédia et Télécommunication.[2]

1.2 Cadre général du projet

Avant l'arrivée de la virtualisation, un opérateur souhaitant exécuter deux systèmes d'exploitation, par exemple Windows et Linux, devait exécuter deux systèmes d'exploitation complets et basculer entre eux. Comme un seul pouvait être utilisé à la fois, il utilisait toute la puissance du matériel.
Dans certains cas, notamment lorsque l'ordinateur n'est pas si puissant et que les tâches sont très exigeantes, certains opérateurs préfèrent encore utiliser le double démarrage. De même, si l'opérateur travaille au sein d'un même système d'exploitation pendant de longues périodes, la puissance disponible lors d'un démarrage double peut être préférable. Toutefois, lorsque l'opérateur doit souvent passer d'une application à l'autre sur différents systèmes d'exploitation, les machines virtuelles sont beaucoup plus pratiques.

1.2.1 Étude de l'existant

1.2.1.1 La virtualisation et son importance

La virtualisation est le processus de création d'une représentation virtuelle, telle que des applications virtuelles, des serveurs, du stockage et des réseaux. C'est le moyen le plus efficace pour réduire les dépenses informatiques tout en améliorant l'efficacité et l'agilité des entreprises de toutes tailles.[3]

1.2.1.2 Les machines virtuelles

Un système informatique virtuel est appelé «machine virtuelle» (VM) : un conteneur de logiciels étroitement isolé avec un système d'exploitation et une application. Chaque machine virtuelle autonome est complètement indépendante. Placer plusieurs ordinateurs virtuels sur un même ordinateur permet à plusieurs systèmes d'exploitation et applications de s'exécuter sur un seul serveur physique, ou «hôte».
Les machines virtuelles sont de plus en plus courantes avec l'évolution de la technologie de virtualisation. Elles sont souvent créées pour effectuer certaines tâches différentes de celles exécutées dans un environnement hôte. Ces machines sont implémentées par des méthodes d'émulation logicielle ou des techniques de virtualisation matérielles. Selon leur utilisation et leur niveau de correspondance avec n'importe quel ordinateur physique, les machines virtuelles peuvent être divisées en deux catégories :

— Machines virtuelles système : plate-forme système prenant en charge le partage des ressources physiques de l'ordinateur hôte entre plusieurs machines virtuelles, chacune exécutant sa propre copie du système d'exploitation. La technique de virtualisation est fournie par une couche logicielle appelée hyperviseur, qui peut s'exécuter soit sur du matériel nu, soit sur un système d'exploitation.

— Processus de machine virtuelle : conçu pour fournir un environnement de programmation indépendant de la plate-forme qui masque les informations du matériel ou du système d'exploitation sous-jacent et permet l'exécution du programme de la même manière sur n'importe quelle plate-forme.[4]

1.2.1.3 Buildroot

Buildroot est un ensemble de Makefiles et de correctifs qui automatise et simplifie le processus de création d'un environnement Linux complet et amorçable pour un système embarqué, tout en ayant recours à la cross-compilation pour permettre la création de plusieurs plateformes cibles sur un système de développement unique basé sur Linux. Buildroot peut automatiquement créer la chaîne d'outils de la corss-compilation requise, créer un système de fichiers racine, compiler une image du noyau Linux et générer un Boot Loader pour le système intégré cible, ou effectuer une combinaison indépendante de ces étapes.[5]

1.2.2 Critique de l'existant

1.2.2.1 Crtitique des machines virtuelles

Les machines virtuelles peuvent utiliser beaucoup de ressources système. Chaque ordinateur virtuel exécute non seulement une copie entière du système d'exploitation, mais aussi une copie virtuelle de tout le matériel que le système d'exploitation est supposé exécuter. Cela ajoute rapidement beaucoup de cycles de RAM et de processeur. C'est toujours économique par rapport à l'exécution d'ordinateurs distincts, mais pour certaines applications, cela est considéré excessif.

1.2.2.2 Critique du Buildroot

La simplicité du Buildroot implique la nécessité d'effectuer une rude personnalisation dans le but de configurer la génération Buildroot pour notre application.
Sans oublier le fait que les options de configuration sont stockées dans un même seul fichier, ce qui nous oblige d'apporter nos modifications pour plusieurs plateformes, au cas nous en utilisons plusieurs.
Toute modification du fichier de configuration du système nécessite également une reconstruction complète de tous les packages.

1.2.3 Solutions proposées

1.2.3.1 Docker

Docker est un outil créé pour faciliter la création, le déploiement et l'exécution des applications à l'aide de conteneurs. Les conteneurs nous permettent de conditionner une application avec toutes les pièces dont nous avons besoin, telles que des bibliothèques et autres dépendances, et de l'expédier dans un package unique. Ainsi, grâce au conteneur, nous pouvons être sûrs que l'application s'exécutera sur toute autre machine Linux, quels que soient les paramètres personnalisés de cette machine susceptibles de différer de la machine utilisée pour l'écriture et le test du code.[6]

D'une certaine manière, Docker est assimilable à une machine virtuelle. Mais contrairement à celle-ci, plutôt que de créer un système d'exploitation virtuel complet, Docker permet aux applications d'utiliser le même noyau Linux que le système de la machine hôte. Cela améliore considérablement les performances et réduit la taille de l'application.
Docker est surtout open source. Cela signifie que n'importe qui peut contribuer à Docker et

l'étendre pour répondre à ses besoins s'il a besoin davantage fonctionnalités qui ne sont pas disponibles ou prêtes à l'emploi.

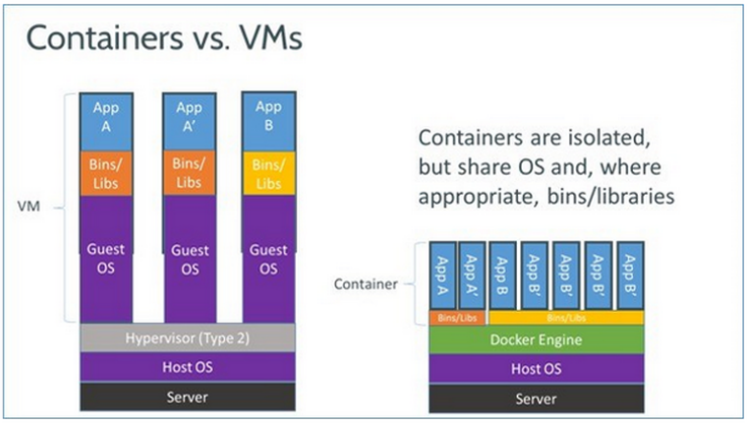


FIGURE 1.1 – Comparaison entre les Conteneurs de Docker et les Machines Virtuelles [7]

1.2.3.2 Le Projet Yocto

Le projet Yocto est un projet de collaboration open source fournissant des outils, des modèles, et des méthodes pour contribuer à la créations de systèmes personnalisés basés sur Linux, quelle que soit l'architecture matérielle. Yocto est basé sur le projet OpenEmbedded. Il est basé sur un ensemble de recettes et des fichiers de configuration utilisés pour créer une distribution Linux adaptée aux besoins spécifiques de l'utilisateur. Yocto permet de compiler tous les éléments nécessaires pour utiliser la cible qui sont :

— Bootloader
— Kernel
— Système de fichiers racine (rootfs)
— Toolchaine
— Software development kit (SDK).[8]

	Yocto	Buildroot
Avantages	-Grande communauté qui contribue dans le projet -Flexibilité de personnalisation, y compris la création des nouvelles couches pour prendre en charge votre propre matériel ainsi que la définition des priorités. -Le support d'appareil le plus large de tous les fabricants de matériel. -Possibilité de créer un SDK pour les clients -Bonne documentation et formation supporté par la fondation linux. -Libération périodique de nouvelles versions.	-Simplicité d'utilisation. -La production du système de fichiers racines le plus petit possible ainsi que le temps de démarrage le plus rapide. -Simplicité de personnalisation. -Production du système de fichiers racine le plus petit possible ainsi que le temps de démarrage le plus rapide. -Communauté active et publication périodique des nouvelles versions tous les 3 mois.
Inconvénients	-Plus de temps et d'efforts pour l'installer correctement.	-Beaucoup de problèmes de compilation et de débogage. -Peut être installé sur une grande variété de systèmes, mais en réalité ne fonctionne bien qu'avec une distribution à la fine pointe.

FIGURE 1.2 – Comparaison entre Yocto et Buildroot

Conclusion

Ce chapitre a détaillé l'étape primordiale de l'étude de tout type de projet qui est l'étude et la critique de l'existant nous permettant ainsi de nous justifier par rapport à nos choix.
Le chapitre d'après comportera les détails portant sur la conception.

Chapitre 2

Spécification des besoins et conception

Chapitre met l'accent sur l'étape de l'analyse des besoins de ce projet, à savoir les besoins fonctionnels et non fonctionnels.
Nous préciserons ensuite les acteurs qui peuvent profiter de notre produit. Et pour finir quelques diagrammes seront présentés pour une meilleure compréhension des caractéristiques de notre projet.

2.1 Spécification et analyse des besoins

Listons tout d'abord les fonctionnalités que notre système doit permettre à ses différents acteurs.

2.1.1 Identification des acteurs

— Le principal acteur est le développeur, celui qui va continuer la génération de l'image et sa customisation, et finira probablement de développer une application sur cette image.

— Un client Docker, étant un acteur secondaire, peut à tout moment télécharger l'image déjà existante dans le Dockerhub.

2.1.2 Les besoins fonctionnels

Ce sont les fonctionnalités basiques et indispensables pour que nous puissions déclarer notre projet réussi. Finalement, notre image doit permettre à l'utilisateur de :

— **Pouvoir ajouter et customiser l'image avec ses propres couches :** après la fin de la génération de l'image du système d'exploitation, l'utilisateur doit pouvoir ajouter facilement n'importe quel type de couche qu'il veut.

— **Tester l'image sur la cible :** L'utilisateur peut ainsi "flasher" sa carte cible avec le système d'exploitation créé et le tester sur un écran.



FIGURE 2.1 – L'image sur l'écran lié à la cible

— **Créer et développer sa propre application :** L'utilisateur peut par exemple ajouter une couche iotivity et développer toute une application IOT en utilisant Qt Creator à titre d'exemple.
— **Pousser son image et/ou son application dans le Dockerhub :** Puisque l(image est générée sous Docker, l'acteur peut l'envoyer vers le Dockerhub en guise de sauvegarde et de partage.
— **Tirer l'image et la télécharger à partir du Dockerhub :** Le client, quelque soit sa distribution de système d'exploitation doit pouvoir télécharger cette image et en profiter sur sa machine.

2.1.3 Les besoins non fonctionnels

— **Disponibilité :** Le container en question, l'image et ses répertoires doivent être disponibles et accessibles à n'importe quel moment.
— **Robustesse :** En travaillant dans des containers nous garantissons cette qualité.

2.1.4 Diagramme de séquences système

La figure qui suit représente le diagramme de séquences système de notre projet. Il détaille les scénarios nominaux et basiques qui peuvent avoir lieu en exploitant notre système.
En effet, une fois l'image est générée, un développeur peut la customiser en ajoutant une couche et en la configurant pour ensuite commencer à développer l'application de son choix. Il peut également voir à quoi ressemble son nouveau système d'exploitation sur un écran lié à sa carte cible. En devenant client Docker il peut la sauvegarder au sein du Dockerhub, pour que n'importe quel autre client Docker peut la télécharger et la déployer sur sa machine.

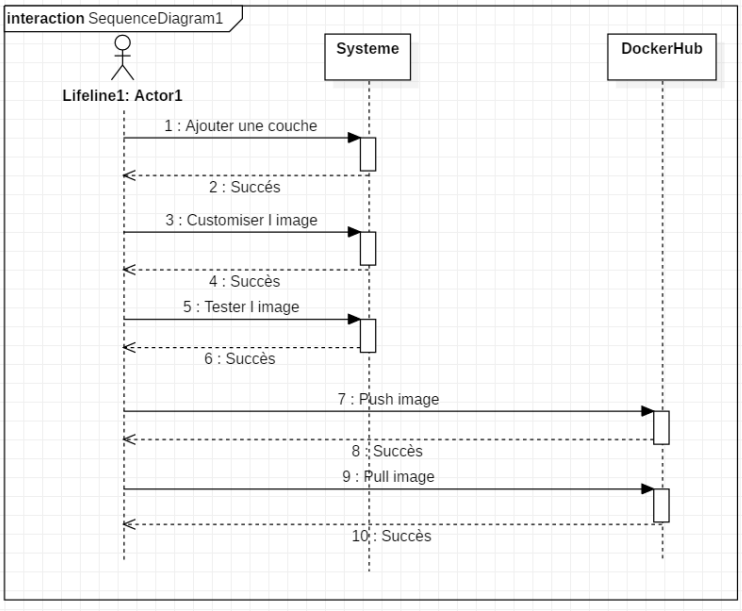


FIGURE 2.2 – Diagramme Séquences-Système

2.2 Conception

Cette section sera dédiée à la spécification de l'architecture de notre solution de base qui est Docker, ainsi que la modélisation du dynamisme de l'une des tâches en ayant recours au diagramme d'activité.

2.2.1 Architecture de Docker

Docker est la plateforme de conteneur la plus populaire. Il permet de créer, de partager et d'exécuter des applications dans les Docker Containers. Docker fait en sorte de séparer les applications qui sont en cours d'exécution de l'infrastructure. Ce qui nous permet d'accélérer considérablement le processus de livraison des applications. Cette plateforme a complètement basculé le mode de développement des applications. Le diagramme ci-dessous, est un aperçu de haut niveau de l'architecture de Docker :

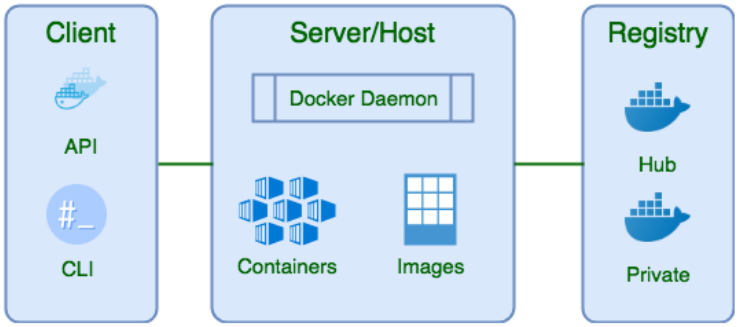


FIGURE 2.3 – Architecture Docker [2]

Docker possède donc une architecture de type client/serveur :

— **Docker Server :** Il s'agit d'un service exécuté en tant que deamon dans un système d'exploitation. Ce service est responsable du téléchargement, de la construction et de l'exécution des conteneurs.

— **Docker Container :** C'est un outil Command-Line Interface qui est responsable de la communication avec les serveurs Docker.

2.2.2 Diagramme d'activité du workflow "se connecter, push et pull à partir du Dockerhub"

Pour pouvoir profiter du Dockerhub, un client doit se connecter via le site du Dockerhub ainsi que via le terminal, créer sa propre repository sur le site, la commande "docker push" lui permet de sauvegarder son image ou application dans le Dockerhub, et "docker pull" lui donne la possibilité de la télécharger à n'importe quel moment sur n'importe quelle machine. Ceci est décrit par le diagramme qui suit :

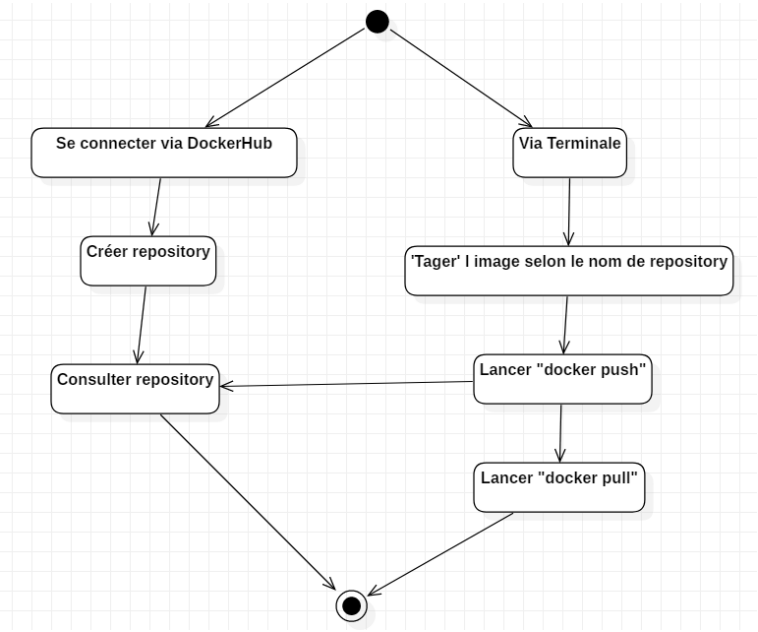


FIGURE 2.4 – Diagramme d'activité

Conclusion

Les acteurs de notre système ont été identifiés dans ce chapitre, donc les besoins fonctionnels et non fonctionnels basiques et auxquels doit répondre le système ont été mises au point, ce qui nous a permis de décrire globalement notre système à l'aide d'un diagramme séquences système et spécialement les scénarios du "push" et "pull" dans le Dockerhub via le diagramme d'activité.
Le troisième et le dernier chapitre comportera les détails reliés à la réalisation et la génération du projet entier.

Chapitre 3

Réalisation

Nous avons au dernier chapitre où nous détaillerons les étapes de réalisation de notre projet.

Environnement matériel

Au long de la réalisation de ce projet, nous avons eu recours au matériel suivant :

3.0.1 PC DELL

— Processeur : Intel® Core™ i7-6700 CPU @ 3.40GHz 8

— Mémoire : 8 Gio

3.0.2 PhyBOARD-Mira i.MX 6 Full Featured

— Système d'exploitation : Linux 4.1.18

— Distribution : Yocto 1.8 (fido)

— Processeur : i.MX 6Quad

— Mémoire : 1GB RAM 64 Bit



FIGURE 3.1 – Le kit PhyBOARD-Mira i.MX 6 Full Featured

3.1 Environnement logiciel

3.1.1 Ubuntu 14.04 LTS

Cet OS basé Linux nous a permis de générer et de customiser notre propre image Linux pour la cible phytec-mira-imx6 sous l'environnement Yocto.

3.1.1.1 Ubuntu 16.04 LTS

Encore un OS basé Linux, utilisé cette fois-ci de profiter de la plateforme Docker pour développer, déployer et d'exécuter notre image Linux dans un container indépendamment de notre machine hôte.

3.1.1.2 Le projet Yocto

Dans notre projet, Yocto nous permet de créer notre propre image "phytec-qt5demo-image" qui n'est d'autre qu'une distribution personnalisée de Linux basée sur Ubuntu 14.04 LTS.

3.1.1.3 Docker

Comme notre travail est réalisé sous Docker, l'étape du "bitbake" sera faite à l'intérieur du container, ce dernier étant créé sur Ubuntu 16.04 LTS à l'aide d'un Dockerfile personnalisé et basé sur Ubuntu 14.04.

3.2 Travail réalisé

3.2.1 Génération et customisation d'une image Linux pour la cible phytec-mira-imx6 sous l'environnent Yocto

Avant de commencer, il faut tout d'abord installer les "Software Packages" nécessaires à savoir :

— Les packages qui vont nous permettre de générer notre image sur un "headless system" qui est un système capable d'opérer sans moniteur, sans interface graphique ni périphériques. Leur installation est déclenchée suite à cette commande :

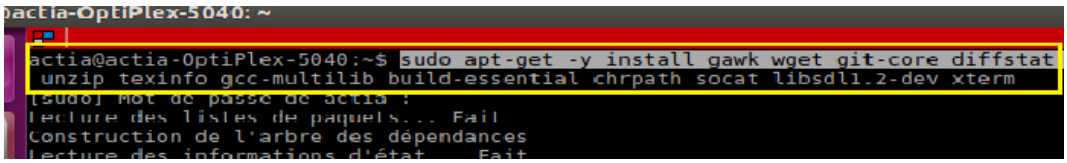


FIGURE 3.2 – headless system packages

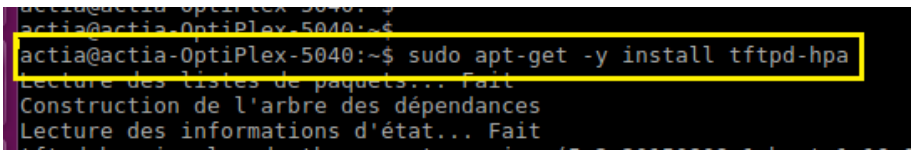


FIGURE 3.3 – tftp-hpa

— Le support de la carte PhyBOARD est basé sur Git, un logiciel qui a besoin que nous lui fournissons certaines informations, étant des utilisateurs, pour qu'il puisse identifier quel changement effectué par quel utilisateur. Pour ce faire, nous avons configuré un

nom d'utilisateur ainsi que son adresse e-mail, exporté le proxy et forcé l'utilisation du protocol "https" pour les liens qui utilisent le protocol "git". Ces informations peuvent être aperçue grâce à cette commande :

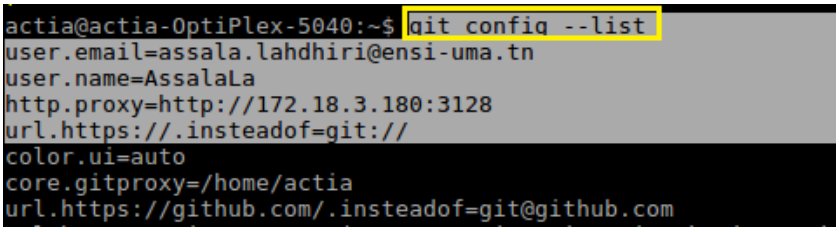


FIGURE 3.4 – git config

— Comme nous travaillons sous un proxy, exporter ce dernier pour tout protocol est une tâche indispensable. Cette opération est accomplie en tapant des commandes de la forme suivante :

export http_proxy="http ://usrname :passwd@host :port"

Pour vérifier que le proxy est bien exporté :

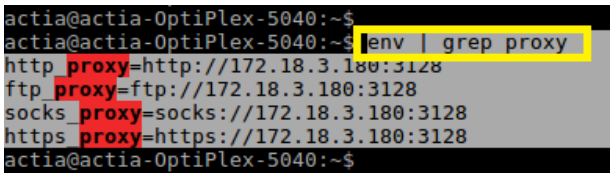


FIGURE 3.5 – proxy

Après avoir effectué les configurations nécessaires, il est temps de passer au sérieux :

— Tout d'abord nous téléchargeons le script phyLinux depuis le site "phytec.de" tout en définissant les privilèges d'exécution :



FIGURE 3.6 – wget phyLinux

— chmod a+x phyLinux pour le rendre exécutable, avant de lancer le script avec ./phyLinux init :

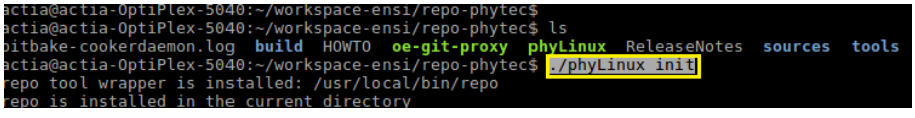


FIGURE 3.7 – phyLinux init

— Nous travaillons avec la carte phyBOARD-Mira i.MX 6, nous choisissons donc :

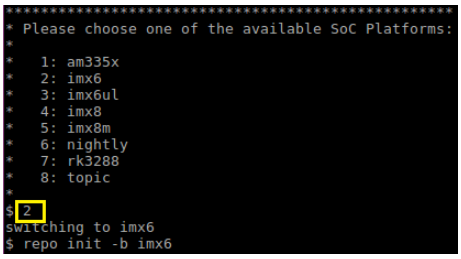


FIGURE 3.8 – imx6

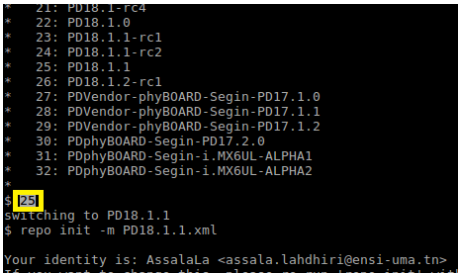


FIGURE 3.9 – PD18.1.1

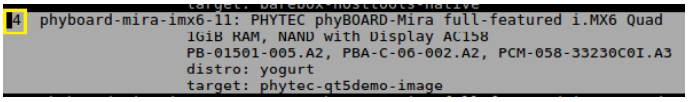


FIGURE 3.10 – phyboard-mira-imx6-11

3.2.2 Sous Docker

Les tâches qui suivent pour la génération de l'image seront réalisées sous Docker, dans un Docker container.
Ce dernier est créé à l'aide d'un Dockerfile qui spécifie la recette qui décrira l'image Docker dont nous avons besoin.
La création du container en question ne dépend que de la commande :

docker image build -t nom_container .

Ne pas oublier le "." à la fin de la commande !

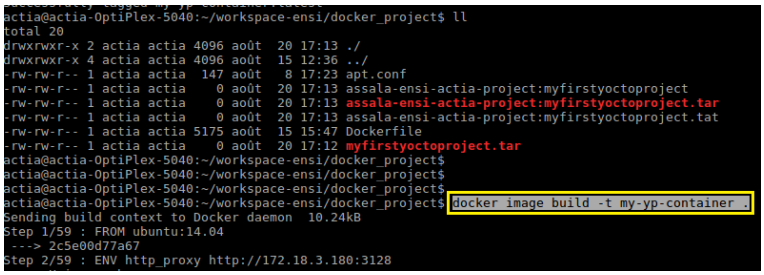


FIGURE 3.11 – docker build

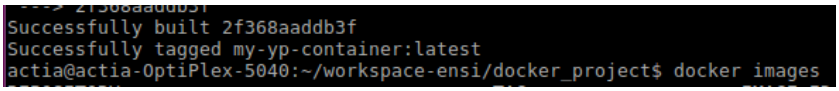


FIGURE 3.12 – successfully built

La commande *docker images* nous permet de percevoir tous nos containers et plusieurs informations à leurs propos, dont la plus importante est l'IMAGE ID.

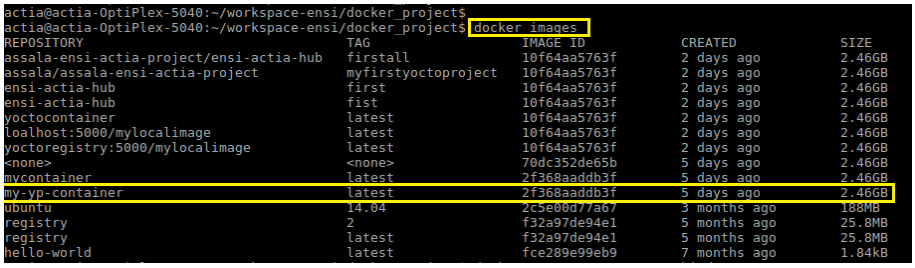


FIGURE 3.13 – docker images

Nous pouvons désormais nous connecter au container à l'aide de l'ID de ce dernier, mais, étant "root" (–user 0) pour pouvoir configurer le mot de passe du user "actia" et donc utiliser les commandes qui nécessitent "sudo" pour s'exécuter.

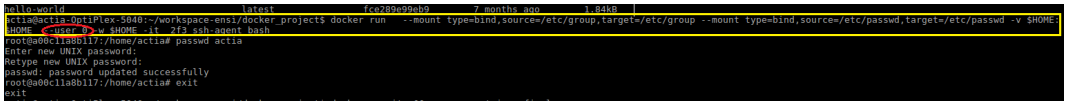


FIGURE 3.14 – docker run root

Une fois le mot de passe configuré, nous quittons le container en tapant exit, nous enregistrons les modifications apportées au container avec *docker commit* et le nouvel ID.

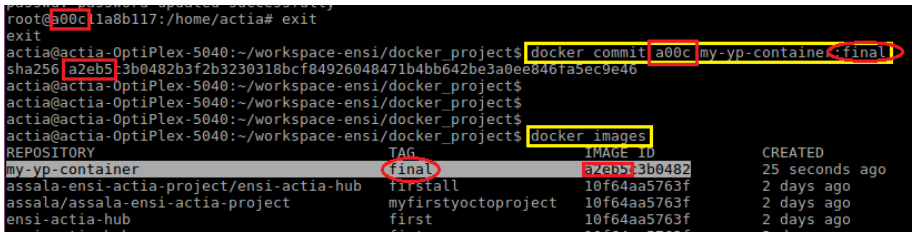


FIGURE 3.15 – docker commit

Connectons-nous maintenant en tant que user : actia



FIGURE 3.16 – docker run user

Nous nous rendons dans le répertoire spécifique à ce projet dans la machine hôte, depuis le container où nous trouvons le fichier *oe-init-build-env* , nous pouvons donc "sourcer" en ayant recours à la commande . *sources/poky/oe-init-build-env* l'environnement en guise de préparation pour le "bitbake".

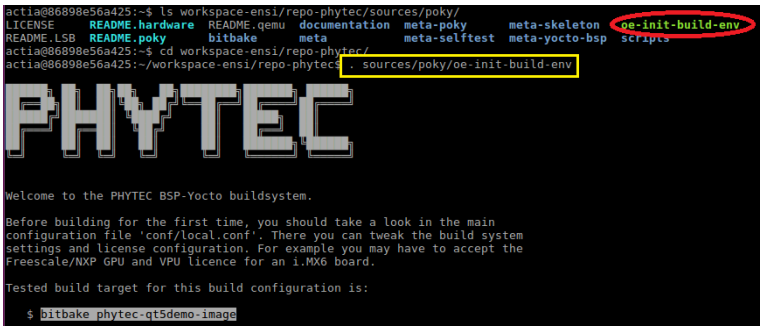


FIGURE 3.17 – . sources

L'environnement est prêt et nous nous trouvons dans le nouveau sous-répertoire "build", "bitbakons" ou "cuissons" notre recette phytec-qt5demo-image !

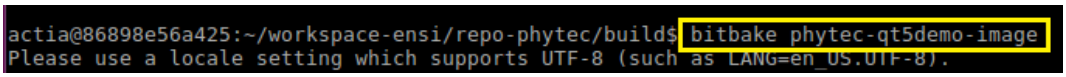


FIGURE 3.18 – bitbake

Nous faisons face à cette erreur :

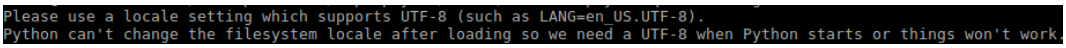


FIGURE 3.19 – UTF-8 erreur

Que nous corrigeons à l'aide de cette liste de commandes :

— *sudo apt-get install locales*
— *sudo dpkg-reconfigure locales*
— *sudo locale-gen en_US.UTF-8*
— *sudo update-locale LC_ALL=en_US.UTF-8*
— *export LANG=en_US.UTF-8*

Et le problème est résolu !
Nous relançons alors la commande *bitbake phytec-qt5demo-image*.

Une erreur surgit qui cause l'échec du bitbake : "ERROR : Nothing PROVIDES <firmware_name> error". Pour la corriger :
Cherchons l'emplacement de ce firmware : *grep -rni <firmware_name>* Ouvrons le fichier en question :
sudo gedit /workspace_ensi/../../file_where_firmware
Commentons la ligne qui exploite ce firmware.
Si jamais une erreur qui tourne autour la couche "RAUC" apparaît, dans notre cas, nous pouvons nous en passer de cette couche donc :

— Cherchons où se manifestent les configurations spécifiques à cette couche :
grep -rni rauc ../sources/meta-yogurt/
Et ouvrons le fichier.
— Dans la partie du fichier qui traite les "RDEPENDS_PN", nous effaçons tout sauf "rauc-update-usb".

Le bitbake peut ainsi avoir lieu et s'achever avec succès :

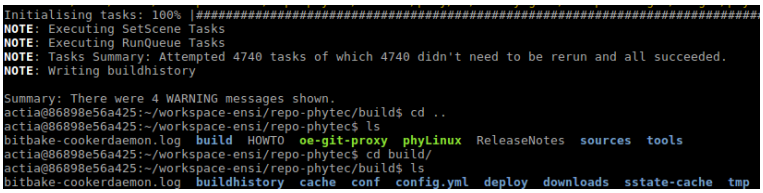


FIGURE 3.20 – succès bitbake

Nous pouvons apercevoir les nouveaux répertoires créés nous menant finalement vers notre images **phyboard-mira-imx6-11** enfin générée :



FIGURE 3.21 – phyboard-mira-imx6-11 générée

Notre image est donc générée et prête. Il est alors temps de la "pusher" dans le Docker Hub. Nous avons créé un compte sur le site ainsi que notre propre repository :

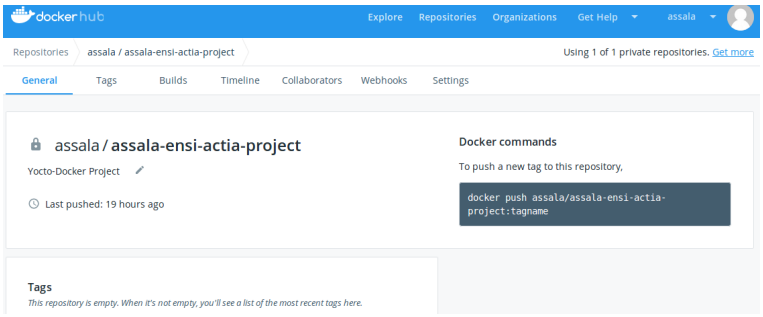


FIGURE 3.22 – repository vide

Connectons-nous au Docker hub également depuis le terminal :

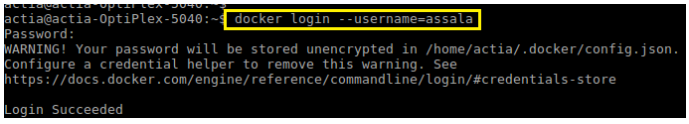


FIGURE 3.23 – docker login

Nous utilisons la commande *docker tag IMAGE ID repo :tag* pour changer le repository de notre image selon le nom de notre Docker hub repository, donc : **assala/assala-ensi-actia-project**

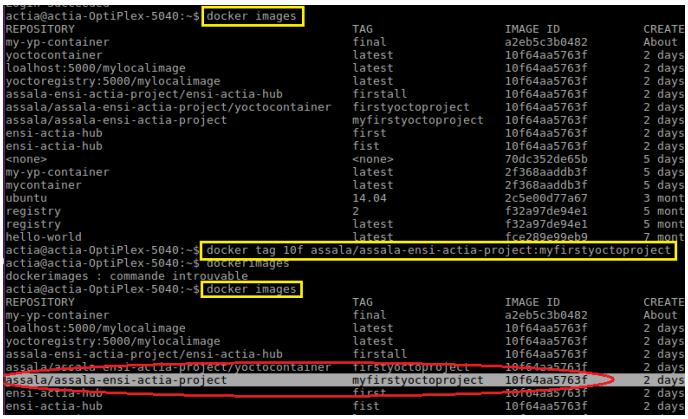


FIGURE 3.24 – docker tag

Nous lançons de suite la commande *docker push* pour copier notre image dans la repository **assala/assala-ensi-actia-project** du Docker hub :

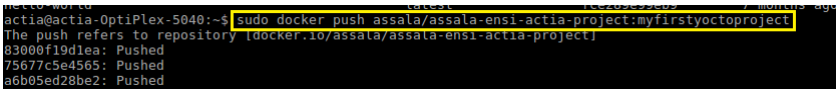


FIGURE 3.25 – docker push



FIGURE 3.26 – pushing

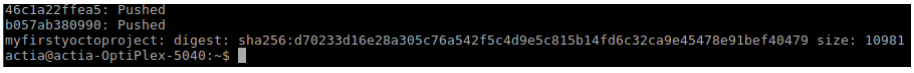


FIGURE 3.27 – successfully pushed

Consultons notre repository, où nous trouvons bel est bien notre image :

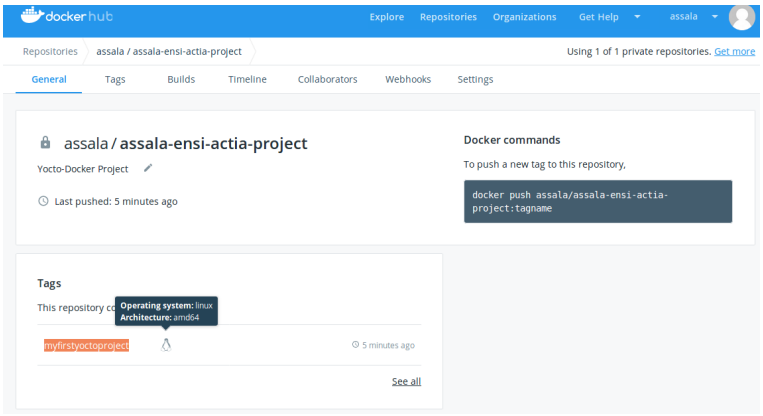


FIGURE 3.28 – repository myfirstproject

N'importe qui, quelque soit sa distribution ou son système d'exploitation, peut désormai récupérer notre image, en s'identifiant et en tapant *docker pull assala/assala-ensi-actia-project :myfirstproject*

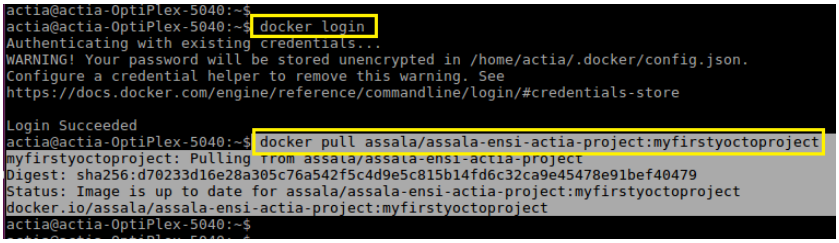


FIGURE 3.29 – docker pull

Conclusion

Tout au long de ce chapitre, nous avons présenté en détail toutes les commandes et les étapes par lesquelles nous somme passés pour arriver finalement à réaliser nos objectifs.

Conclusion Générale et Perspectives

Docker permet l'isolement des applications dans des conteneurs avec des instructions qui indiquent exactement ce dont elles ont besoin pour survivre et qui peuvent être transférées d'une machine à l'autre en toute simplicité.
C'est exactement la raison pour laquelle que nous avons opté pour cette solution afin de garantir un maximum de compatibilité de notre système avec différentes machines et distributions.
Le fait d'utiliser le projet Yocto et de générer notre image dans un Docker container ont fait en sorte que le développeur peut à tout moment décider et customiser ce produit avec la couche qu'il souhaite, enrichissant ainsi les performances de sa cible tout en la testant grâce à une simple commande. Il peut également avoir la conscience tranquille vis à vis du sauvegarde de son produit car l'image est générée dans un Docker container, donc elle peut être facilement envoyer vers le fameux Dockerhub, d'où elle est également aisément téléchargeable par n'importe quel client Docker.
Lors de ce projet nous avions pour but de prouver que toute cible peut ne pas se limiter à ses fonctionnalités par défaut tout en ayant recours à une plateforme de construction dont les avantages n'ont fait d'elle qu'un succès incontestable.
Malgré les erreurs et les troubles que nous avons rencontré nous jugeons notre travail satisfaisant et nous en sommes fières mais au pas au point de le considérer idéal.
Si la durée du stage était plus longue, nous aurions pu enrichir notre produit en lui créant notre propre Docker registry qui peut jouer le rôle d'un serveur local, ceci à titre d'exemple.

Bibliographie

[1] : https ://fr.wikipedia.org/wiki/Actia$_{G}$rouphttps : $//fr.wikipedia.org/wiki/Actia_{G}roup(dernireconsu$
$le02Juillet2019)$

[2] : $http : //ardia.com.tn/index.php/qui - sommes - nous/presentationhttp : //ardia.com.tn/index.php$
$le02Juillet2019)$

[3] : $https : //fr.wikipedia.org/wiki/Virtualisationhttps : //fr.wikipedia.org/wiki/Virtualisation(derni$
$le06Juillet2019)$

[4] : $https : //www.lemagit.fr/definition/Machine - Virtuelle - VMhttps : //www.lemagit.fr/definit$
$le06Juillet2019)$

[5] : https ://linux.developpez.com/tutoriels/embarque-buildroot/https ://linux.developpez.com/tu
buildroot/ (dernière consultation : le 09 Juillet 2019)

[6] : https ://fr.wikipedia.org/wiki/Docker$_{(}logiciel)https : //fr.wikipedia.org/wiki/Docker_{(}logiciel)($
$le10Juillet2019)$

[7] : https ://techburst.io/containers-vs-vms-what-is-the-battle-all-about-a17fb2162826https ://techbu
vs-vms-what-is-the-battle-all-about-a17fb2162826 (dernière consultation : le 15 Juillet 2019)

[8] : https ://linuxfr.org/news/yocto-ce-n-est-pas-un-systeme-embarque-maishttps ://linuxfr.org/news/
ce-n-est-pas-un-systeme-embarque-mais (dernière consultation : le 15 Juillet 2019)

[9] : https ://subscription.packtpub.com/book/application$_{d}evelopment/9781788992329/1/ch01lvl1sec13/$
$container-architecturehttps : //subscription.packtpub.com/book/application_{d}evelopment/9781788992329/1/$
$le22Juillet2019)$